An Online Kernel Method Based on Orthogonal Projection Reflecting Multiple Tasks

潮 旭

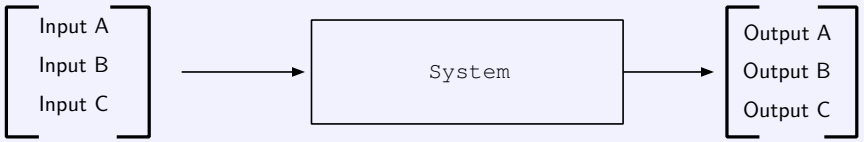
慶應義塾大学理工学部電子工学科
湯川研究室
学籍番号：61102827

卒論発表会
2016 年 2 月 5 日

ベクトル値信号処理

ベクトル値信号処理とは

- 複数の信号を一つのベクトル信号として処理
  $\longrightarrow$ 複数のシステムをベクトル値関数として同時推定
- モチベーション $\longrightarrow$ タスクの関係性を考慮した推定
  (Multi-task learning) による推定性能向上



ベクトル値信号処理の例

1. **ある点での東西南北 4 方向の風力**を**気候情報**から推定 [1]
   $\longrightarrow$ 出力 : $[W_{west}, W_{east}, W_{south}, W_{north}]$,　入力 : [気温, 湿度 , ...]
2. **複数の皮膚病の容態の変化**を**幾つかの要因**から推定 [2]
   $\longrightarrow$ 出力 : [皮膚病 A, 皮膚病 B, 皮膚病 C, ...],
   入力 : [血圧, 体温, 心拍数 , ...]

[1] Tobar, *et al* 2014　[2] Audiffren *et al* 2013

ベクトル値信号処理の課題

- 複雑なシステムの推定
  $\longrightarrow$ 非線形システム
- 逐次的に得られるデータの処理
  $\longrightarrow$ 株価データ、気象データなど
- 計算量の削減
- タスク間の関係性の抽出方法 (Multi-task learning)

Table: Misclassification Error [3]

Task	Data	SVM	Indiv-SVM	Multi SVM
50	20	41.97	29.86	**28.72**
100	20	41.41	29.86	**28.3**
200	20	40.08	29.86	**27.79**

[3] Theodoros Evgeniou, Charles A Micchelli, and Massimiliano Pontil,
"Learning multiple tasks with kernel methods," in Journal of Machine Learning Research, 2005, pp. 615 - 637.

本論文では

提案法

1. カーネル法 → **非線形推定可能**
2. NLMS ベースの学習方法 → **オンライン学習**
3. Coherence により辞書に入れる入力を選別 + Selective update
   → **性能を保持しつつ計算量削減**
4. 提案モデルから導かれる Multi-task Kernel による学習
   → **Multi-task Learning (タスク間の影響考慮)**

アルゴリズム	辞書	逐次性	Multi-task	非線形
Multi-task kernel [3], [4]	×	×	o	×
ONORMA [2]	×	o	o	o
MKLMS [1], Multi-regression [5]	o	o	×	o
提案手法	o	o	o	o

[1] Tobar *et al.*, 2014 [2] Audiffren *et al.*, 2013 [3] Evgeniou *et al.*, 2005 [4] Argyriou *et al.*, 2008 [5] Slavakis *et al.*, 2012

問題設定

以下のベクトル値関数 $F \in \boldsymbol{\mathcal{H}}$ の推定を考える.

$$\boldsymbol{d} = F(\boldsymbol{U}) = \begin{bmatrix} f_1(\boldsymbol{u}_1) \\ \vdots \\ f_\tau(\boldsymbol{u}_\tau) \end{bmatrix} + \begin{bmatrix} \eta_1 \\ \vdots \\ \eta_\tau \end{bmatrix}, \tag{1}$$

$$\begin{bmatrix} \eta_1 \\ \vdots \\ \eta_\tau \end{bmatrix} \sim i.i.d.\, \mathcal{N}\left(\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix} \right),$$

ここで $\boldsymbol{\mathcal{H}}$ は $\mathcal{U}$ から $\mathcal{D}$ へのヒルベルト空間.

f_t の推定：タスク t

- 入力空間：$\boldsymbol{U} = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\tau]^\mathsf{T} \in \mathcal{U} := \mathbb{R}^{L\tau}$, $\boldsymbol{u}_t \in \mathbb{R}^L$
- 出力空間：$\boldsymbol{d} = [d_1, \ldots, d_\tau]^\mathsf{T} \in \mathcal{D} := \mathbb{R}^\tau$

既存モデル

- **f_t を線形と仮定した場合** ($t = 1, \ldots, \tau$)
  F の推定量としてタスク間の影響力を操作するパラメータ $\mu \in [0, 1/\tau)$ による線形 Multi-task Learning Model が提案されている [1].

$$\hat{\boldsymbol{g}}(\boldsymbol{U}) := \boldsymbol{M}\tilde{\boldsymbol{g}}(\boldsymbol{U}) \tag{2}$$

$$\tilde{\boldsymbol{g}}(\boldsymbol{U}) := \begin{bmatrix} \tilde{g}_1(\boldsymbol{u}_1) \\ \vdots \\ \tilde{g}_\tau(\boldsymbol{u}_\tau) \end{bmatrix}, \ \tilde{g}_t(\boldsymbol{u}_t) := \boldsymbol{\omega}_t^\mathsf{T}\boldsymbol{u}_t, \tag{3}$$

$$\boldsymbol{M} := \begin{bmatrix} 1-(\tau-1)\mu & \mu & \ldots & \mu \\ \mu & 1-(\tau-1)\mu & \ldots & \mu \\ \vdots & \vdots & \ddots & \vdots \\ \mu & \mu & \ldots & 1-(\tau-1)\mu \end{bmatrix}. \tag{4}$$

μ が小さい $\longrightarrow$ タスク間の影響力は弱い (学習は別々)
μ が大きい $\longrightarrow$ タスク間の影響力は強い (Multi-task learning)

[1] Evgeniou *et al.*, 2005

提案モデル

- **非線形の一般的な Multi-task Learning Model の設計**
  $\longrightarrow$ (2) のモデルを非線形化する.

$$\hat{F}(\boldsymbol{U}) := \boldsymbol{M}\boldsymbol{\varphi}(\boldsymbol{U}) \tag{5}$$

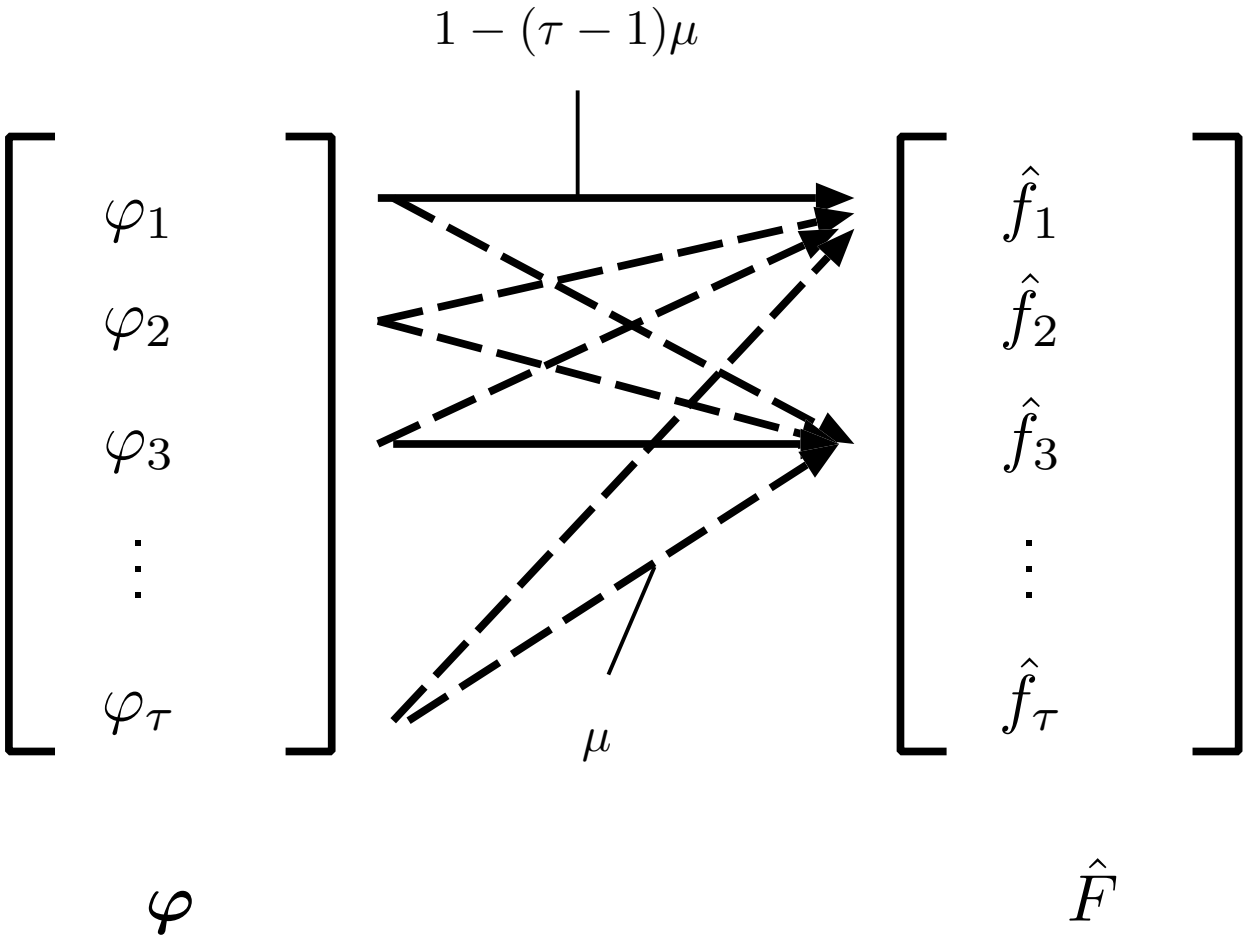
$$\boldsymbol{\varphi}(\boldsymbol{U}) := \begin{bmatrix} \varphi_1(\boldsymbol{u}_1) \\ \vdots \\ \varphi_\tau(\boldsymbol{u}_\tau) \end{bmatrix} \tag{6}$$

ここで φ_t は

$$\varphi_t \in \mathcal{H} : \mathbb{R}^L \to \mathbb{R},$$
$$\varphi_t : \boldsymbol{u}_t \mapsto \varphi_t(\boldsymbol{u}_t) = \langle \kappa(\boldsymbol{u}_t, \cdot), \varphi_t \rangle_{\mathcal{H}}$$

である. ここで κ はカーネル.

提案モデル



- $\boldsymbol{\varphi}$: 独立なタスクごとのシステム
- $\hat{F}$: $\boldsymbol{\varphi}$ の各要素が結合したシステム

辞書設計

(5) のモデルで観測値に対し**瞬時誤差 0 になるような超平面**を定義

$$\mathcal{V}_n := \left\{ \hat{F} \in \boldsymbol{\mathcal{H}} | \boldsymbol{d}_n = \hat{F}(\boldsymbol{U}_n) = \boldsymbol{M}\boldsymbol{\varphi}(\boldsymbol{U}_n) \right\}. \tag{7}$$

関数空間

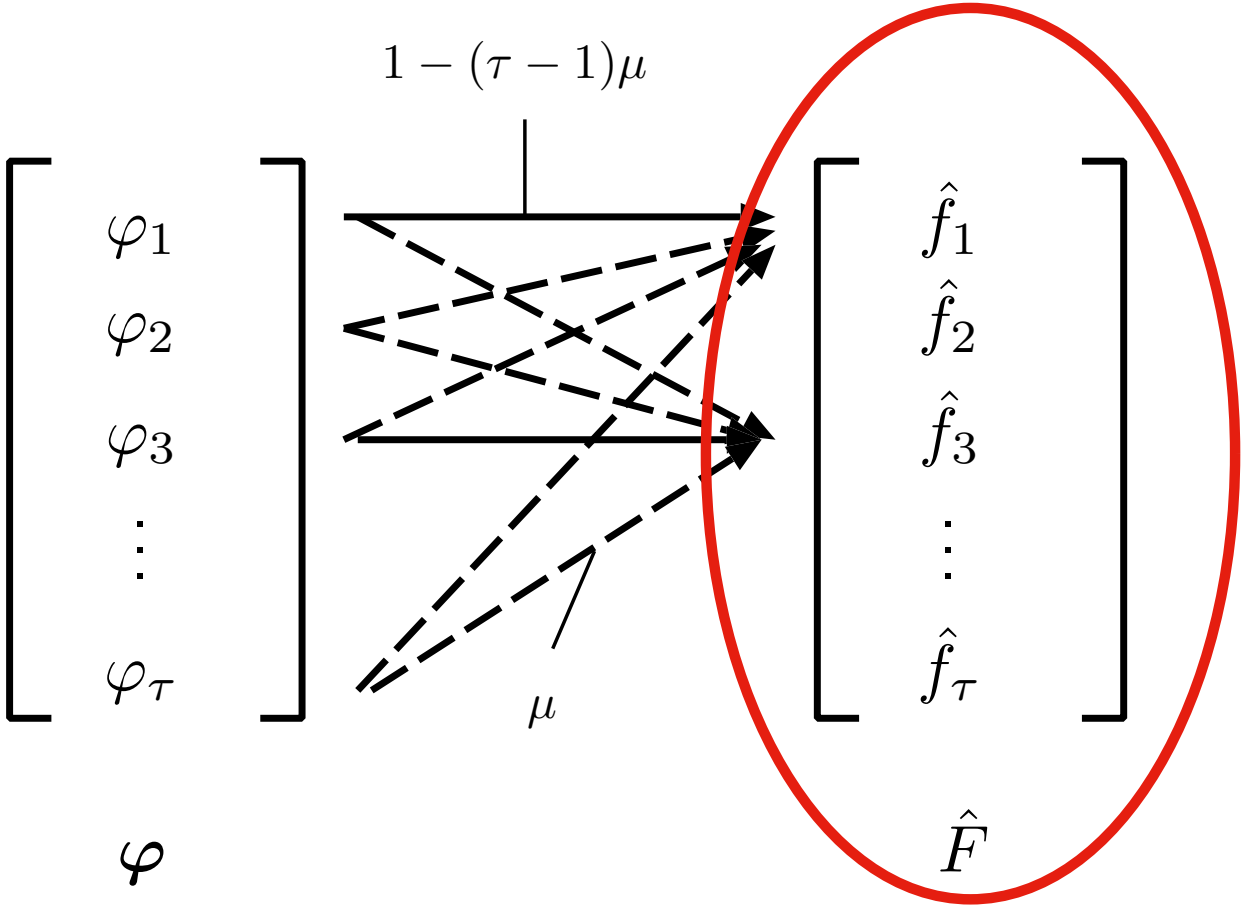
$$\tilde{\boldsymbol{\mathcal{H}}} := \left\{ \boldsymbol{\varphi} | \boldsymbol{\varphi} = \boldsymbol{M}^{-1}\hat{F}, \hat{F} \in \boldsymbol{\mathcal{H}} \right\} \tag{8}$$

での**瞬時誤差 0 の超平面**を考えると

$$\begin{aligned} \tilde{\mathcal{V}}_n &:= \left\{ \boldsymbol{\varphi} \in \tilde{\boldsymbol{\mathcal{H}}} | \boldsymbol{M}^{-1}\boldsymbol{d}_n = \boldsymbol{\varphi}(\boldsymbol{U}_n) \right\} \\ &= \left\{ \boldsymbol{\varphi} \in \tilde{\boldsymbol{\mathcal{H}}} | \tilde{d}_{t,n} = \varphi_t(\boldsymbol{u}_{t,n}) = \langle \varphi_t, \kappa(\boldsymbol{u}_{t,n}, \cdot) \rangle_{\mathcal{H}}, \ \forall\ t = 1, \ldots, \tau \right\}. \end{aligned} \tag{9}$$

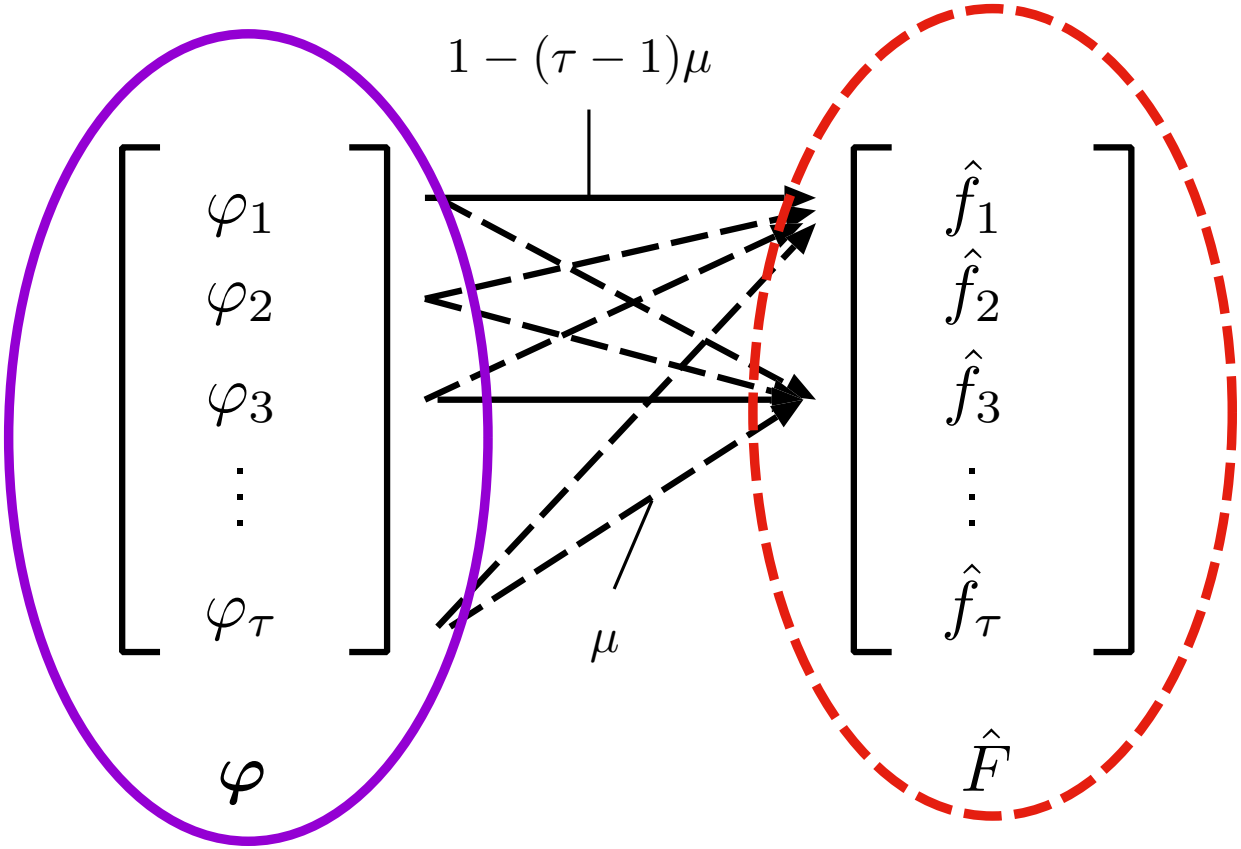
- $M^{-1}\boldsymbol{d}_n = [\tilde{d}_{1,n}, \ldots, \tilde{d}_{\tau,n}]^{\mathsf{T}}$

辞書構築の概念図



- 直接 $\hat{F}$ の空間で射影するのではなく.....

辞書構築の概念図



- 直接 $\hat{F}$ の空間で射影するのではなく $\boldsymbol{\varphi}$ の空間で射影
- $\boldsymbol{\varphi}$: 独立なタスクごとのシステム

辞書設計

$\tilde{\mathcal{V}}_n$ への射影方向の導出

$$\mathcal{L}_{t,n} := \left\{ \varphi \in \mathcal{H} | \langle \varphi, \kappa(\boldsymbol{u}_{t,n}, \cdot) \rangle_{\mathcal{H}} = \tilde{d}_n \right\}. \tag{10}$$

を用いると $\tilde{\mathcal{V}}_n$ への $\boldsymbol{\varphi}_{n-1}$ の射影方向は

$$P_{\tilde{\mathcal{V}}_n}(\boldsymbol{\varphi}_{n-1}) - \boldsymbol{\varphi}_{n-1} = \begin{bmatrix} P_{\mathcal{L}_{1,n}}(\varphi_{1,n-1}) \\ \vdots \\ P_{\mathcal{L}_{\tau,n}}(\varphi_{\tau,n-1}) \end{bmatrix} = \begin{bmatrix} \xi_{1,n}\kappa(\boldsymbol{u}_{1,n}, \cdot) \\ \vdots \\ \xi_{\tau,n}\kappa(\boldsymbol{u}_{\tau,n}, \cdot) \end{bmatrix} \tag{11}$$

と導かれる．ここで

$$\xi_{t,n} = \frac{\tilde{d}_n - \varphi_{t,n}(\boldsymbol{u}_{t,n})}{||\kappa(\boldsymbol{u}_{t,n}, \cdot)||^2_{\mathcal{H}}}, \tag{12}$$

for $t = 1, \ldots, \tau$.

Multi-task kernel の導出

$\mathcal{V}_n$ への写像の導出

- $\hat{F}_{n-1}$ の $\mathcal{V}_n$ への写像を以下のように定義する.

$$M_{\mathcal{V}_n}(\hat{F}_{n-1}) := \boldsymbol{M}\left(P_{\tilde{\mathcal{V}}_n}(\boldsymbol{\varphi}_{n-1}) - \boldsymbol{\varphi}_{n-1}\right). \tag{13}$$

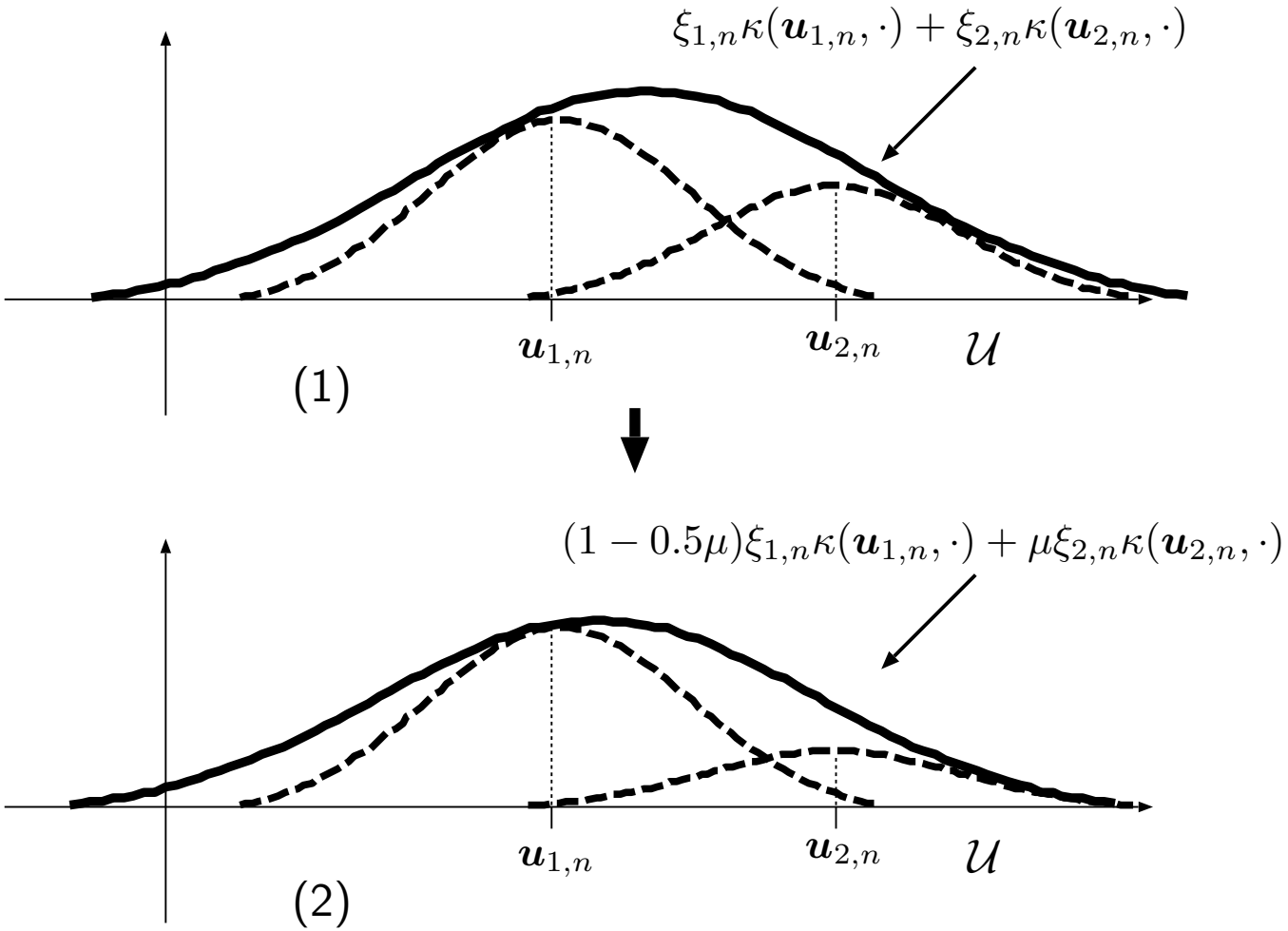
- (13) に (11) を代入すると

$$M_{\mathcal{V}_n}(\hat{F}_{n-1}) = \boldsymbol{M}\begin{bmatrix} \xi_{1,n}\kappa(\boldsymbol{u}_{1,n},\cdot) \\ \vdots \\ \xi_{\tau,n}\kappa(\boldsymbol{u}_{\tau,n},\cdot) \end{bmatrix} = \begin{bmatrix} K_{1,n}(\boldsymbol{U}_n,\cdot) \\ \vdots \\ K_{\tau,n}(\boldsymbol{U}_n,\cdot) \end{bmatrix} \tag{14}$$

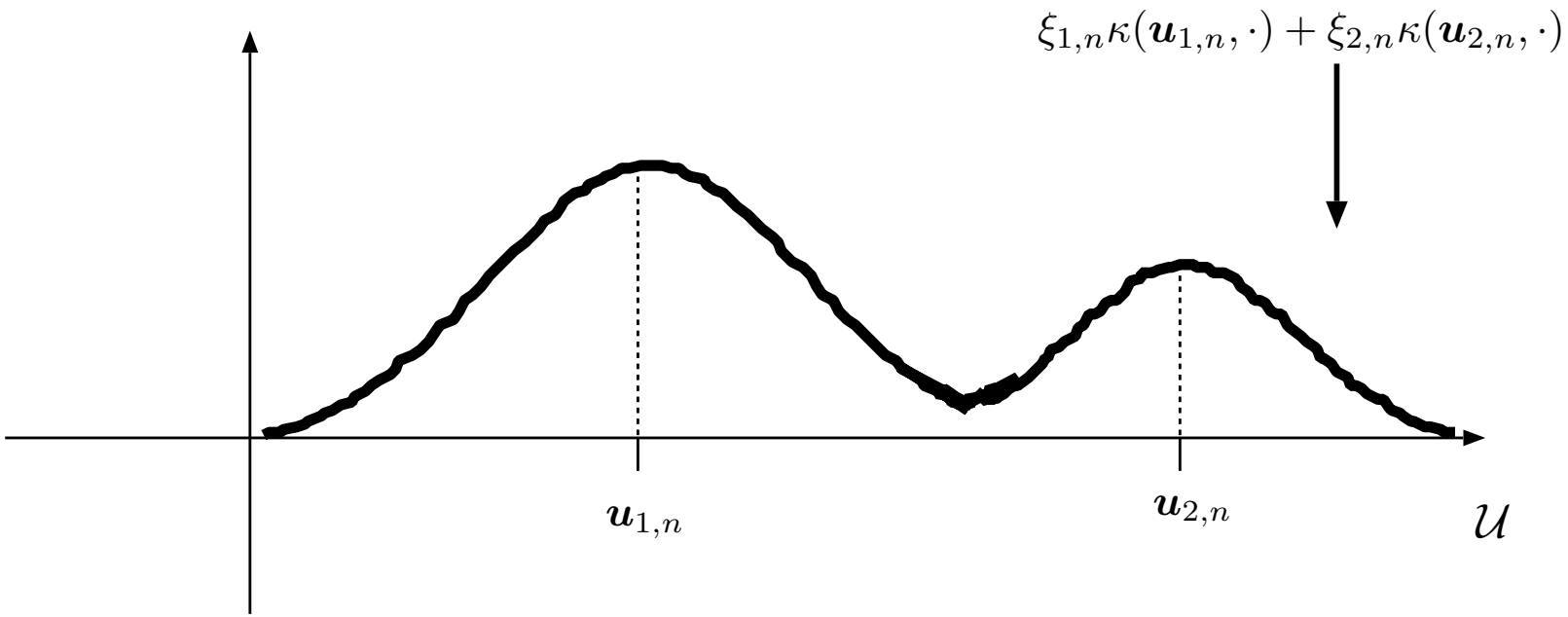
$$\begin{aligned} K_{t,n}(\boldsymbol{U}_n,\boldsymbol{V}_n) &= \{1-(\tau-1)\mu\}\xi_{t,n}\kappa(\boldsymbol{u}_{t,n},\boldsymbol{v}_{t,n}) \\ &\quad + \mu\sum_{a=1,a\neq t}^{\tau}\xi_{a,n}\kappa(\boldsymbol{u}_{a,n},\boldsymbol{v}_{t,n}) \end{aligned} \tag{15}$$

- $K_{t,n}$ を **Multi-task kernel** と定義する.

Multi-task kernel の概念図



Multi-task kernel の概念図 (補足)



Multi-task kernel による辞書設計

- 最終的に F_n は逐次的に更新される係数 $h^t_{j,n} \in \mathbb{R}$ により

$$\hat{F}_n(\boldsymbol{U}_n) = \begin{bmatrix} \hat{f}_{1,n}(\boldsymbol{U}_n) \\ \vdots \\ \hat{f}_{\tau,n}(\boldsymbol{U}_n) \end{bmatrix} = \begin{bmatrix} \sum_{j \in \mathcal{J}^1_n} h^1_{j,n} K_{1,j}(\boldsymbol{U}_j, \boldsymbol{U}_n) \\ \vdots \\ \sum_{j \in \mathcal{J}^\tau_n} h^\tau_{j,n} K_{\tau,j}(\boldsymbol{U}_j, \boldsymbol{U}_n) \end{bmatrix} \tag{16}$$

  の形で推定される.
- 辞書空間

$$\begin{aligned} \mathcal{M}_n &:= \mathcal{M}_{1,n} \times \cdots \times \mathcal{M}_{\tau,n} \\ \mathcal{M}_{t,n} &:= \operatorname{span}\left\{K_{t,j}(\boldsymbol{U}_j, \cdot)\right\}_{j \in \mathcal{J}^t_n} \end{aligned} \tag{17}$$

  ここで $\mathcal{J}^t_n = \{j^{(t,n)}_1, \ldots, j^{(t,n)}_{r^t_n}\} \subset \{0, 1, \ldots, n-1\}$ はタスク t の辞書の index set (r^t_n は辞書サイズ).
- Coherence 基準により辞書に入れるかどうかを判断

フィルタ更新方法

フィルタ係数の選択更新 [6]

係数更新する辞書を制限する $\longrightarrow$ 計算コスト削減

- 入力に対する Coherence で更新係数を Q 個まで選択 $(\mathcal{I}_n^t := \{l_1^{(t,n)}, \ldots, l_Q^{(t,n)}\} \subset \mathcal{J}_n^t)$
- この場合更新は

$$\tilde{\mathcal{M}}_n := \tilde{\mathcal{M}}_{1,n} \times \cdots \times \tilde{\mathcal{M}}_{\tau,n} \subset \mathcal{M}_n \tag{18}$$
$$\tilde{\mathcal{M}}_{t,n} := \mathrm{span}\left\{K_{t,l}(\boldsymbol{U}_l, \cdot)\right\}_{l \in \mathcal{I}_n^t}.$$

に制限される．すなわち $\hat{F}_{n+1}$ は Π_n 上の関数．

$$\Pi_n := \hat{F}_n + \tilde{\mathcal{M}}_n = \left\{\hat{F}_n + G : G \in \tilde{\mathcal{M}}_n\right\} \tag{19}$$

[6] M.Takizawa *et al.*, 2015

直交射影の導出

- Π_n への直交射影によるフィルタ更新を行う

フィルタ更新式

初期値 $\hat{F}_0 := 0$ とし、$\hat{F}_n$ を以下のルールで更新する.

$$\hat{F}_{n+1} := \hat{F}_n + \lambda(P_{\mathcal{V}_n \cap \Pi_n}(\hat{F}_n) - \hat{F}_n), \tag{20}$$

$\lambda \in (0, 2)$: ステップサイズ

射影 $P_{\mathcal{V}_n \cap \Pi_n}(F_n)$ は以下の様に求まる.

$$P_{\mathcal{V}_n \cap \Pi_n}(\hat{F}_n) = \hat{F}_n + \sum_{t=1}^{\tau} \boldsymbol{e}_t \beta_{t,n} P_{\tilde{\mathcal{M}}_{t,n}}(K_{t,n}(\boldsymbol{U}_n, \cdot)) \tag{21}$$

$$\beta_{t,n} = \frac{d_{t,n} - \hat{f}_{t,n}(\boldsymbol{U}_n)}{\sum_{l \in \mathcal{I}_n^t} \alpha_{t,l} K_{t,l}(\boldsymbol{U}_l, \boldsymbol{U}_n)}. \tag{22}$$

$\boldsymbol{e}_t$: t 番目の要素が 1 の単位ベクトル

直交射影の導出

射影 $P_{\tilde{\mathcal{M}}_{t,n}}(K_{t,n}(\boldsymbol{U}_n,\cdot))$ は

$$P_{\tilde{\mathcal{M}}_{t,n}}(K_{t,n}(\boldsymbol{U}_n,\cdot)) = \sum_{l\in\mathcal{I}_n^t} \alpha_{t,l} K_{t,l}(\boldsymbol{U}_l,\cdot), \quad \alpha_{t,l}\in\mathbb{R}. \tag{23}$$

(20),(21),(23) より $F_{n+1} = [f_{1,n+1},\ldots,f_{\tau,n+1}]^\mathsf{T}$ は

$$\hat{f}_{t,n+1} := \sum_{l\in\mathcal{I}_n} (h_{l,n}^t + \lambda\beta_{t,n}\alpha_{t,l}) K_{t,l}(\boldsymbol{U}_l,\cdot) + \sum_{j\in\mathcal{J}_n^t\setminus\mathcal{I}_n} h_{j,n}^t K_{t,j}(\boldsymbol{U}_j,\cdot), \tag{24}$$

となる $(t=1,\ldots,\tau)$.

直交射影の導出

係数ベクトル $\boldsymbol{\alpha}_t = \left[\alpha_{t,l_1^{(t,n)}}, \ldots, \alpha_{t,q^{(t,n)}}\right]^\mathsf{T}$ は以下の様に求まる

$$\boldsymbol{K}_{t,n}\boldsymbol{\alpha}_t = \boldsymbol{p}_{t,n}, \tag{25}$$

ここで

$$\boldsymbol{K}_{t,n} := \begin{bmatrix} K_{t,l_1^{(n)}}(\boldsymbol{U}_{l_1^{(n)}}, \boldsymbol{U}_{l_1^{(n)}}) & \ldots & K_{t,l_1^{(n)}}(\boldsymbol{U}_{l_1^{(n)}}, \boldsymbol{U}_{q^{(t,n)}}) \\ \vdots & \ddots & \vdots \\ K_{t,q^{(t,n)}}(\boldsymbol{U}_{q^{(t,n)}}, \boldsymbol{U}_{l_1^{(n)}}) & \ldots & K_{t,q^{(t,n)}}(\boldsymbol{U}_{q^{(t,n)}}, \boldsymbol{U}_{q^{(t,n)}}) \end{bmatrix} \tag{26}$$

$$\boldsymbol{p}_{t,n} := [K_{t,l_1^{(n)}}(\boldsymbol{U}_{l_1^{(n)}}, \boldsymbol{U}_n), \ldots, (K_{t,q^{(t,n)}}(\boldsymbol{U}_{q^{(t,n)}}, \boldsymbol{U}_n)]^\mathsf{T} \tag{27}$$

$$q^{(t,n)} := \begin{cases} l_{r_n^t}^{(t,n)}, & (\text{if } Q \geq r_n^t) \\ l_Q^{(t,n)}, & (\text{else}). \end{cases} \tag{28}$$

アルゴリズム要約

The Proposed Algorithm

Requirement : step size $\lambda \in [0, 2]$, $\mu \in [0, 1/\tau)$

Initialization : Define $\mathcal{J}_{-1}^t := \phi$ for $t = 1, \ldots, \tau$, calculate M^{-1}.

Filter output : $\hat{F}_n(\boldsymbol{U}_n) := [\hat{f}_1(\boldsymbol{U}_n), \ldots, \hat{f}_\tau(\boldsymbol{U}_n)]^\mathsf{T}$ where

$$f_t(\boldsymbol{U}_n) = \sum_{j \in \mathcal{J}_n^t} h_{j,n}^t K_{t,j}(\boldsymbol{U}_j, \boldsymbol{U}_n).$$

Filter update :

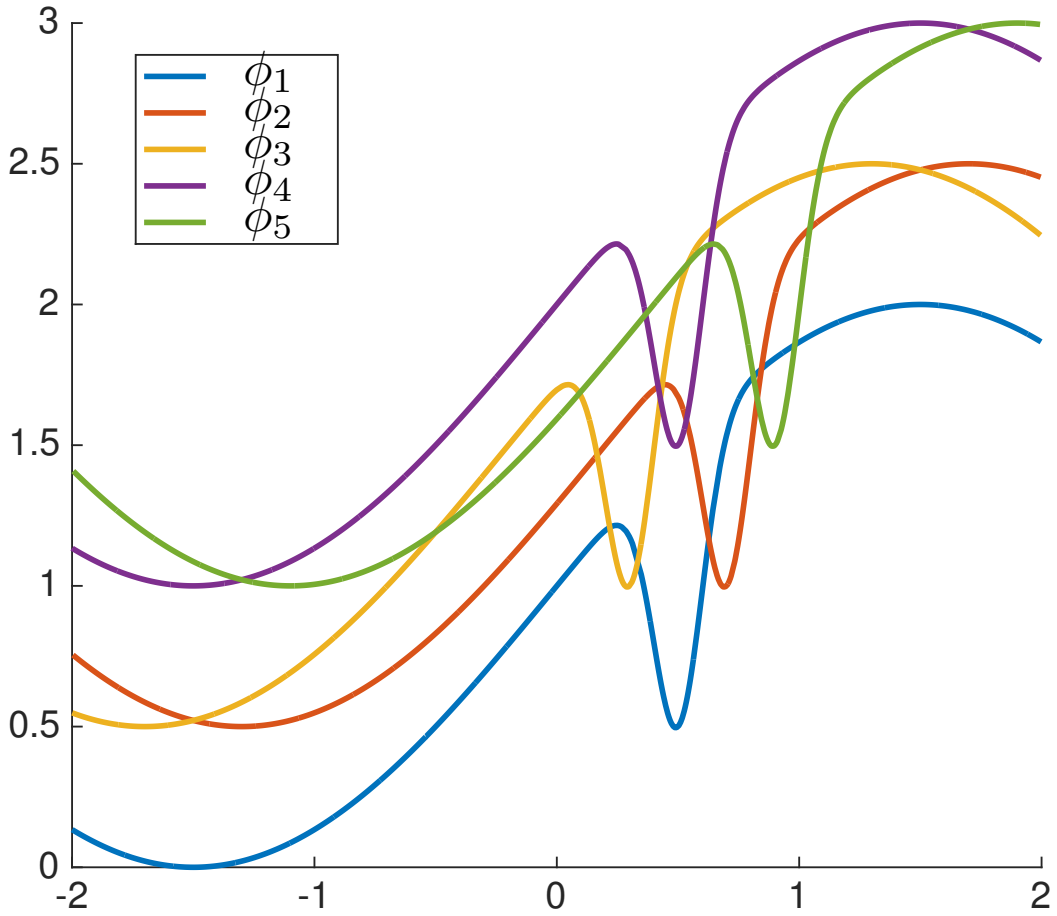
1. Calculate instantaneous error $\boldsymbol{\epsilon}_n \in \mathbb{R}^\tau$ by $\boldsymbol{\epsilon}_n = \boldsymbol{d}_n - \hat{F}_{n-1}(\boldsymbol{U}_n)$.
2. Calculate $\boldsymbol{\xi}_n = [\xi_{1,n}, \ldots, \xi_{\tau,n}]^\mathsf{T} = \boldsymbol{M}^{-1}\boldsymbol{\epsilon}_n / ||\kappa(\boldsymbol{u}, \boldsymbol{u})||$ (see (12)).
3. For $t = 1, \ldots, \tau$,
   - 3.1. Define $\mathcal{J}_n^t$ based on the coherence criterion.
   - 3.2. If $n \in \mathcal{J}_n^t$, let $h_{n,n}^t := 0$.
   - 3.3. If $r_n^t > Q$, define $\mathcal{I}_n^t := \{l_1^{(t,n)}, \ldots, l_Q^{(t,n)}\} \subset \mathcal{J}_n^t$.
     Otherwise, let $\mathcal{I}_n^t := \mathcal{J}_n^t$.
   - 3.4. Compute $\boldsymbol{\alpha}_t$ by (25).
   - 3.5. Compute $\beta_{t,n}$ (22).
   - 3.6. Update the coefficients by $h_{l,n+1}^t := h_{l,n}^t + \lambda \beta_{t,n} \alpha_{t,l}$ for all $l \in \mathcal{I}_n^t$.

数値実験

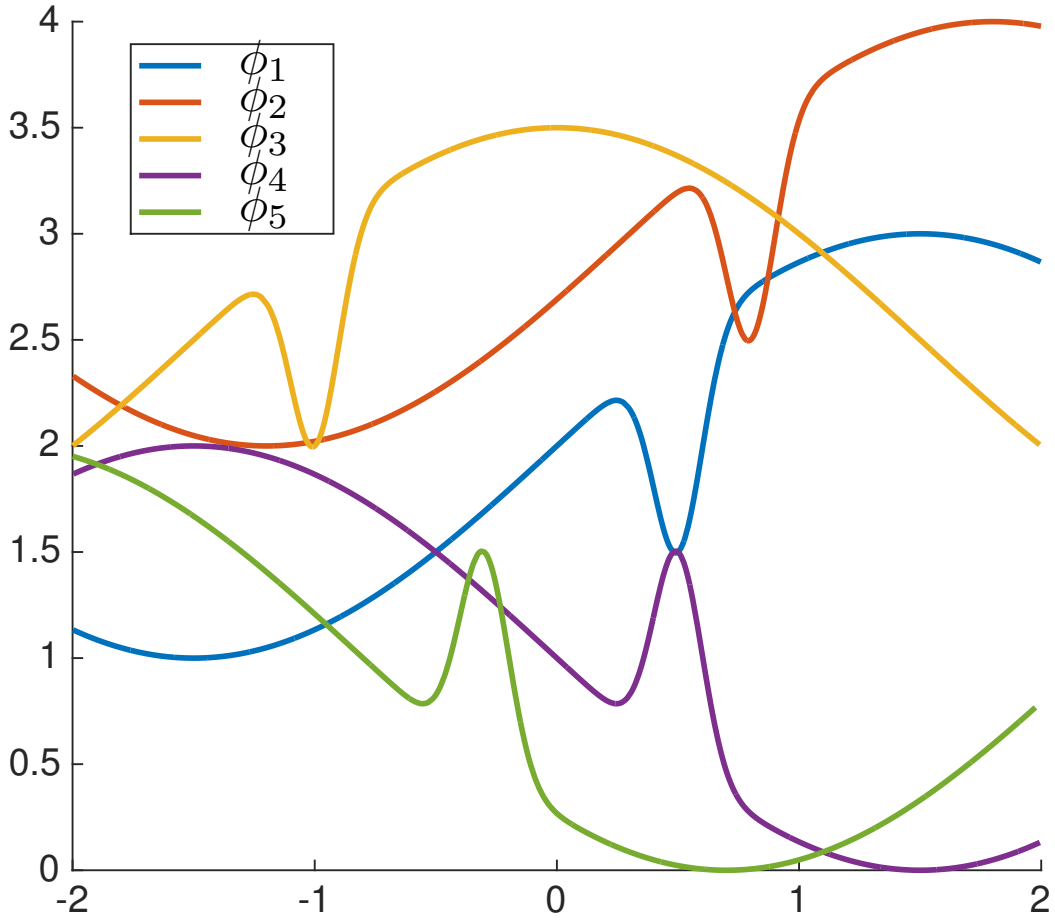
- **目的**
  マルチタスクの効果の検証
- **比較アルゴリズム**
  1. 提案手法 (μ =0.2)
  2. スカラー値関数の推定アルゴリズム HYPASS [6]
     $\longrightarrow$ タスク事に別々に推定 (提案手法の μ=0 に相当)
- **シミュレーション条件**
  (a) タスクが似ている場合
  (b) タスクが似てない場合
- **Multi-task kernel を作る元のカーネル κ について**
  全てガウスカーネル.
- **辞書のセレクト数 Q**
  どちらも 1 .

[6] M.Takizawa *et al.*, 2015

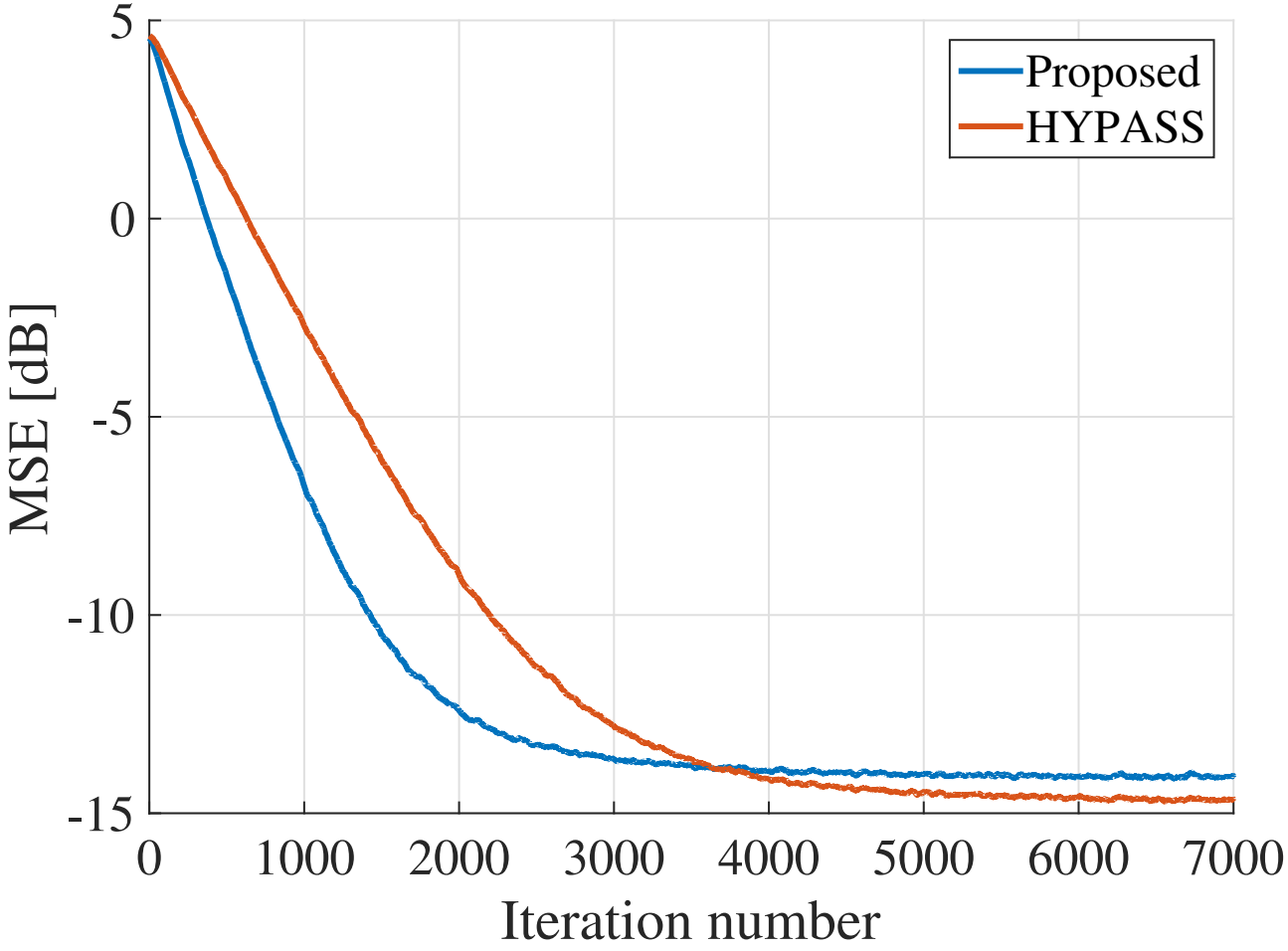
タスクが似ている場合



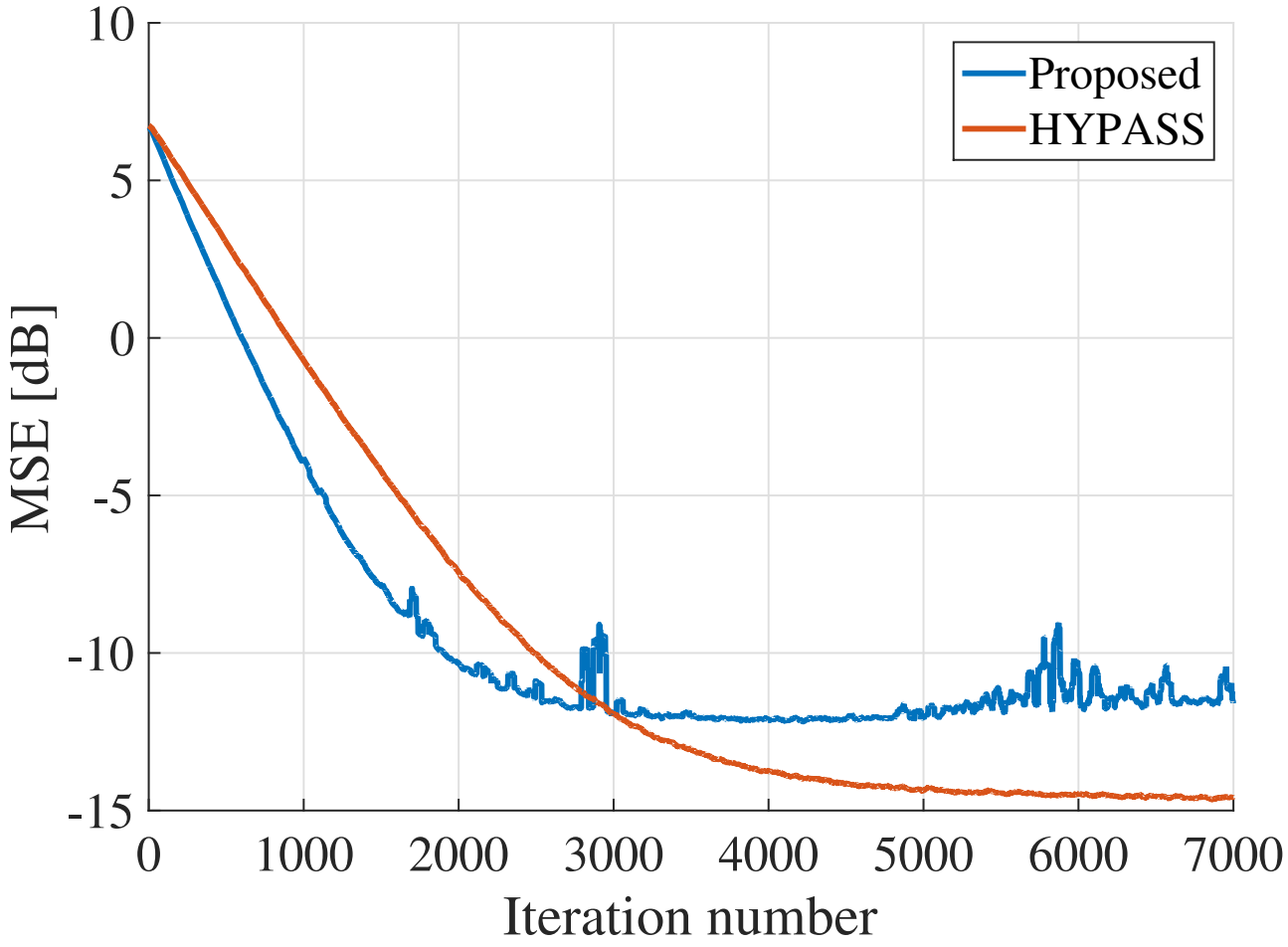
タスクが似ていない場合



タスクが似ている場合の MSE (300 回独立試行平均)

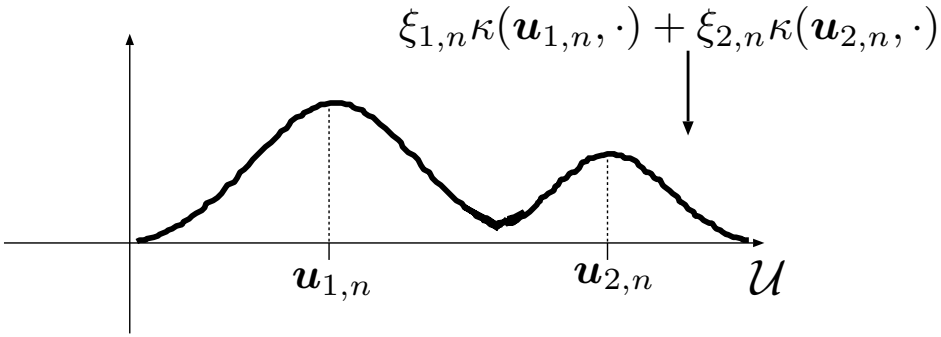


タスクが似ていない場合の MSE (300 回独立試行平均)



考察

- タスクが似ている場合の結果では**初期収束の向上**
  $\longrightarrow$ 提案手法によりタスク間の関係性が上手く推定に反映されている
- タスクが似ていない場合の結果では**推定が不安定**
  $\longrightarrow$ タスクが似ていないため推定結果を悪化させるマルチタスクカーネルが辞書要素に加えられている

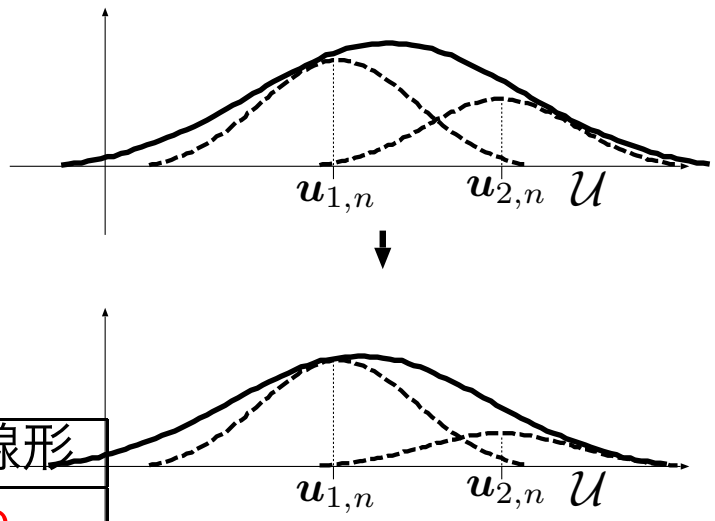


- 初期収束が速い $\longrightarrow$ 以下の条件下で有効である.
  1. **タスクごとのデータが限られている**
  2. **類似タスクが幾つかある**

まとめと展望

まとめ

- 射影法、カーネル法を基に
  Multi-task learning アルゴリズムを提案
- 数値実験より別々に推定するよりも
  初期収束が向上されることを示した

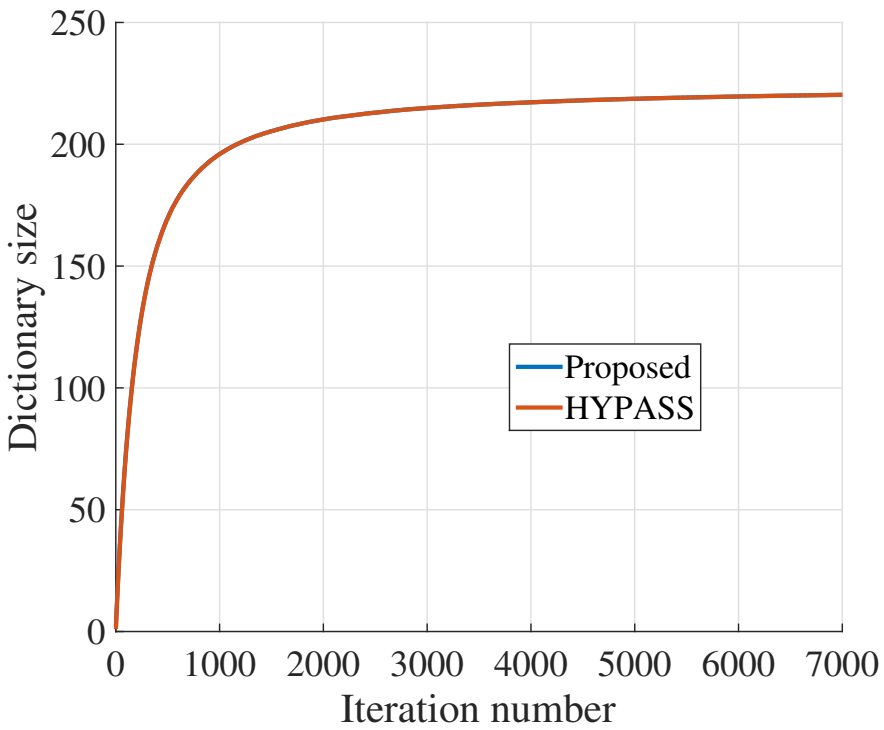


	辞書	逐次性	Multi-task	非線形
提案手法	o	o	o	o

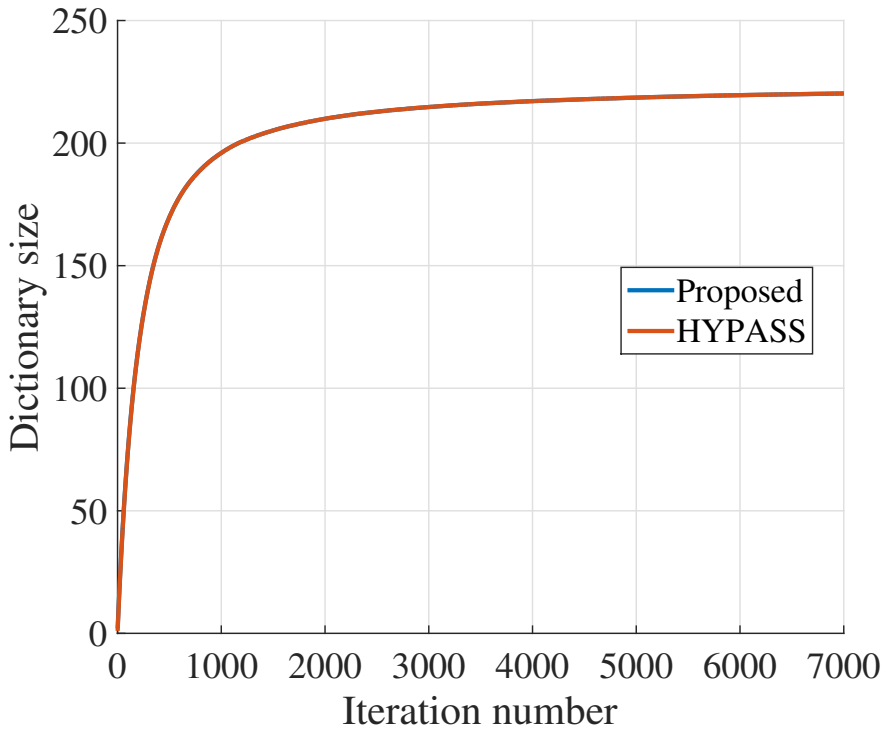
展望

- μ の最適化
  $\longrightarrow$ 逐次的に μ タスクの関係を汲み取って更新して最適化する必要
- 従来手法との比較
- 実データへの応用

Appendix (辞書サイズ)



(a) の辞書サイズの推移



(b) の辞書サイズの推移

Appendix (Complexity)

Complexity (number of multiplications in update per iteration)

- Proposed Algorithm $>$ HYPASS
- 数値実験の条件 $\tau = 5, Q = 1$, では実行時間はほぼ同じ (Q : selective-update size, τ : task size).

Algorithm	Complexity	$\tau = 5, Q = 1$
Proposed	$\{(Q^2 - Q)L\tau/2 + O(Q^3) + Q^2 + 2Q\}\tau + \tau^2$	40
HYPASS	$\{(Q^2 - Q)L/2 + O(Q^3) + Q^2 + 2Q\}\tau$	15

Appendix (Simulation Conditions)

- **Simulation Conditions**

algorithm	HYPASS	Proposed Algorithm
Q	1	1
μ	-	0.2
σ	0.2	0.2
λ	0.1	0.1
ϵ	10^{-4}	10^{-4}
δ	0.9	0.9

Table: Parameters for algorithms

Gaussian white noises are added to observed data.

Appendix (Novelty Criterion)

- **Novelty criterion**

Coherence criterion : $k_t(u_{t,n}, \cdot)$ is regarded to be novel if

$$\max_{i \in \mathcal{J}_{n-1}^t} \exp\left(-\frac{(u_{t,i} - u_{t,n})^2}{2\sigma}\right) < \delta \tag{29}$$

for $\delta := 0.9$.